Part 22: Feature Engineering

Financial Data Science II Updated March 17, 2026

Part 22: Feature Engineering

Data rarely arrive in a form that is immediately suitable for predictive analysis. Feature engineering refers to the construction of variables from the observed data that may be informative for a forecasting or decision problem.

In finance this step is especially important. Signals are weak, relationships change over time, and even small errors in timing can invalidate an analysis. Accordingly, the construction of features should be treated as part of the modelling problem itself, rather than as a routine preprocessing step.

The emphasis in this Part is on two settings.

1. Daily equity prediction using market and accounting data.
2. Short-horizon prediction using limit order book data.

Part 23 will take the next step and consider how such features may be screened, compared, and validated.

Exercise. Give three examples of raw data encountered in finance and explain how each could be transformed into a useful feature for prediction.

Basic Terms

We use the following terms throughout.

A **feature** (or **predictor**) is a variable used to predict some future quantity. A **target** (or **response**) is the quantity to be forecast. A **forecast horizon** is the time between the observation of the feature and the measurement of the target.

In daily equity modelling, one common target is a 5-day forward return. In microstructure analysis, one common target is the next movement of the mid-price. These are different problems and generally require different feature sets.

We also distinguish between:

1. **Clock time:** data aggregated to regular intervals such as one minute or one day.
2. **Event time:** data indexed by market events, such as order submissions, cancellations, or trades.

Finally, **lookahead bias** occurs when a feature contains information that would not have been available at the prediction time. This is one form of **data leakage**, and it can make a poor model look excellent in backtesting.

Exercise. Suppose that you wish to predict a stock's return over the next 5 trading days. What information is valid to use on day t? What information would constitute leakage?

Why Feature Engineering Matters in Finance

Feature engineering matters in any predictive problem, but several aspects of financial data make it especially important here.

1. Financial data are noisy.
2. Relationships are often nonstationary.
3. Timing matters. A feature that is valid at the daily level may be invalid at the intraday level.
4. Useful predictors often arise from economic or market-structure considerations, not from generic transformations alone.

Thus, a modest model with carefully constructed features can outperform a more sophisticated model fitted to weak or misaligned inputs.

Data Sources and Timing

In the examples below we use three broad kinds of information.

1. **Daily market data:** open, high, low, close, adjusted close, and volume.
2. **Accounting data:** quarterly statements used to form rough valuation ratios such as P/E and P/B.
3. **Limit order book data:** event-level message and order book files from LOBSTER.

These sources do not arrive on the same schedule. Market prices are observed continuously. Accounting variables are released with delay. Order book data are recorded in event time rather than at fixed intervals. The timing convention is therefore part of the feature definition.

Example Prediction Tasks

To fix ideas, we consider two examples in this Part.

1. **Daily equity prediction:** use market and accounting data to predict a forward return over several days.
2. **Limit order book prediction:** use LOBSTER message and order book data to predict short-horizon mid-price direction.

The first example emphasizes feature families commonly used in equity prediction. The second emphasizes market microstructure and the difference between event-time data and fixed-interval bars.

Feature Families for Daily Data

The daily example uses the following feature families.

1. Returns and log returns.
2. Momentum over multiple horizons.
3. Rolling volatility.
4. Drawdown.
5. Valuation ratios such as P/E and P/B.

The point of these features is not merely to create many columns. Each predictor should encode an economically or statistically meaningful aspect of the price process.

Feature Families for LOB Data

The LOB example uses the following feature families.

1. Spread and mid-price.
2. Level-1 imbalance.
3. Multi-level depth imbalance.
4. Order flow imbalance (OFI).
5. Microprice deviation.
6. Message intensity and signed volume.

These features summarize short-run pressure from demand and supply at the book. Because the underlying data arrive in event time, one must decide carefully how to aggregate them when building fixed-interval predictors.

Exercise. For each of the two examples above, identify one feature that would be easy to compute but likely uninformative, and one feature that is harder to compute but may be economically meaningful.

Example A: Daily Equity Data

We begin with a standard daily equity example using `yfinance`. The objective is not to argue that the particular features below are optimal. Rather, the objective is to illustrate a systematic procedure for constructing predictors from observed data.

Observed Variables

The raw daily data consist of prices and trading volume. These are observed directly from the market. They are not yet features in the modelling sense.

Derived Variables

From these quantities we may construct returns, rolling volatility, momentum, and drawdown. We may also add accounting-based ratios when statement data are available and when the timing convention is handled carefully.

```
import warnings
warnings.filterwarnings('ignore')

from pathlib import Path
import zipfile

import numpy as np
import pandas as pd
import yfinance as yf

from sklearn.linear_model import LinearRegression, Ridge, LogisticRegression
from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score,
    balanced_accuracy_score,
    roc_auc_score,
)
from sklearn.model_selection import TimeSeriesSplit

from xgboost import XGBRegressor, XGBClassifier
```

```
TICKER = 'AAPL'
START = '2016-01-01'
END = '2026-03-01'

px = yf.download(TICKER, start=START, end=END, auto_adjust=False, progress=False)
if isinstance(px.columns, pd.MultiIndex):
    px.columns = px.columns.get_level_values(0)
px.columns = [str(c).lower().replace(' ', '_') for c in px.columns]
```

```
if 'adj_close' not in px.columns and 'adj close' in px.columns:
    px['adj_close'] = px['adj close']
if 'adj_close' not in px.columns:
    px['adj_close'] = px['close']
px.head()
```

	adj_close	close	high	low	open	volume
Date						
2016-01-04	23.730949	26.337500	26.342501	25.500000	25.652500	270597600
2016-01-05	23.136263	25.677500	26.462500	25.602501	26.437500	223164000
2016-01-06	22.683498	25.174999	25.592501	24.967501	25.139999	273829600
2016-01-07	21.726151	24.112499	25.032499	24.107500	24.670000	324377600
2016-01-08	21.841036	24.240000	24.777500	24.190001	24.637501	283192000

Exercise. Inspect the downloaded data. Which columns are observed directly from the market, and which are already transformed?

Price-Based Features

We begin with a small set of features based only on historical prices.

```
feat = pd.DataFrame(index=px.index)
feat['ret_1d'] = px['adj_close'].pct_change()
feat['log_ret_1d'] = np.log(px['adj_close'] / px['adj_close'].shift(1))

for window in [5, 21, 63, 126]:
    feat[f'mom_{window}d'] = px['adj_close'].pct_change(window)
    feat[f'vol_{window}d'] = feat['ret_1d'].rolling(window).std()

rolling_max = px['adj_close'].cummax()
feat['drawdown'] = px['adj_close'] / rolling_max - 1
feat.tail()
```

	ret_1d	log_ret_1d	mom_5d	vol_5d	mom_21d	vol_21d	mom_63d	vol_63
Date								
2026-02-23	0.006047	0.006029	0.040660	0.016972	0.072797	0.019943	-0.007935	0.01403
2026-02-24	0.022391	0.022144	0.031302	0.014001	0.098188	0.020322	0.023079	0.01427
2026-02-25	0.007680	0.007651	0.037375	0.013775	0.074690	0.019517	0.011038	0.01409
2026-02-26	-0.004668	-0.004679	0.047471	0.010201	0.057829	0.019516	-0.009838	0.01395
2026-02-27	-0.032130	-0.032658	-0.001512	0.020316	0.031146	0.020873	-0.045286	0.01451

Accounting-Based Features

A second class of predictors is based on accounting information. For illustration, we form quarterly estimates of earnings per share and book value per share when the underlying statement data are available. These are then used to construct rough P/E and P/B ratios.

```
def get_financial_table(ticker, attr):
    obj = getattr(ticker, attr, None)
    return obj if isinstance(obj, pd.DataFrame) else pd.DataFrame()


def get_shares_series(ticker):
    shares = None
    for getter in ('get_shares_full', 'get_shares'):
        try:
            shares = getattr(ticker, getter)()
            if shares is not None:
                break
        except Exception:
            continue

    if isinstance(shares, pd.DataFrame) and shares.shape[1] > 0:
        shares = shares.iloc[:, 0]
    if shares is None or not hasattr(shares, 'index'):
        return None

    shares = shares[~shares.index.duplicated(keep='last')]
    shares.index = pd.to_datetime(shares.index, errors='coerce').tz_localize(None)
    return shares.dropna()


def build_quarterly_per_share(statement_df, shares, name_options):
    if statement_df.empty or shares is None or len(shares) == 0:
        return None
```

```
    statement_df = statement_df.T.copy()
    statement_df.index = pd.to_datetime(statement_df.index, errors='coerce').tz_localize(Nor
    statement_df = statement_df.dropna(how='all')

    selected_col = None
    for col in statement_df.columns:
        if any(name in str(col) for name in name_options):
            selected_col = col
            break
    if selected_col is None:
        return None

    series = statement_df[selected_col]
    shares_aligned = shares.reindex(statement_df.index, method='nearest')
    out = (series / shares_aligned).replace([np.inf, -np.inf], np.nan)
    if isinstance(out, pd.DataFrame) and out.shape[1] > 0:
        out = out.iloc[:, 0]
    return out
```

```
def build_fundamental_features(ticker_symbol, price_index, price_series):
    tk = yf.Ticker(ticker_symbol)

    income_q = get_financial_table(tk, 'quarterly_income_stmt')
    if income_q.empty:
        income_q = get_financial_table(tk, 'income_stmt')

    balance_q = get_financial_table(tk, 'quarterly_balance_sheet')
    if balance_q.empty:
        balance_q = get_financial_table(tk, 'balance_sheet')

    shares = get_shares_series(tk)
    fund = pd.DataFrame(index=price_index)

    eps_q = build_quarterly_per_share(income_q, shares, ['Net Income'])
    if eps_q is not None:
        fund['eps_q'] = eps_q.reindex(fund.index, method='ffill')

    bvps_q = build_quarterly_per_share(balance_q, shares, ['Total Stockholder Equity', 'Tota
    if bvps_q is not None:
        fund['bvps_q'] = bvps_q.reindex(fund.index, method='ffill')

    price_series = pd.Series(price_series, index=price_index).astype(float)

    if 'eps_q' in fund.columns:
        eps = pd.Series(fund['eps_q'], index=fund.index).astype(float)
        fund['pe'] = price_series / eps
    if 'bvps_q' in fund.columns:
        bvps = pd.Series(fund['bvps_q'], index=fund.index).astype(float)
        fund['pb'] = price_series / bvps

    return fund.replace([np.inf, -np.inf], np.nan)


fund = build_fundamental_features(TICKER, px.index, px['adj_close'])

features_daily = feat.join(
```

```
    fund[['pe', 'pb']] if {'pe', 'pb'}.intersection(fund.columns) else fund,
    how='left'
)
features_daily = features_daily.dropna(how='all')
features_daily.tail()
```

	ret_1d	log_ret_1d	mom_5d	vol_5d	mom_21d	vol_21d	mom_63d	vol_63
Date								
2026-02-23	0.006047	0.006029	0.040660	0.016972	0.072797	0.019943	-0.007935	0.01403
2026-02-24	0.022391	0.022144	0.031302	0.014001	0.098188	0.020322	0.023079	0.01427
2026-02-25	0.007680	0.007651	0.037375	0.013775	0.074690	0.019517	0.011038	0.01409
2026-02-26	-0.004668	-0.004679	0.047471	0.010201	0.057829	0.019516	-0.009838	0.01395
2026-02-27	-0.032130	-0.032658	-0.001512	0.020316	0.031146	0.020873	-0.045286	0.01451

Constructing the Target

Suppose that P_t denotes a daily adjusted close price. A standard forward return target with horizon h is

$$Y_t = racP_{t+h}P_t - 1.$$

This target uses future prices by construction, but only on the right-hand side of the modelling problem. The features used to predict Y_t must be based only on information available at time t.

```
TARGET_HORIZON = 5

features_daily['target_fwd_ret_5d'] = (
    px['adj_close'].reindex(features_daily.index).shift(-TARGET_HORIZON)
    / px['adj_close'].reindex(features_daily.index)
    - 1
)

model_data_daily = features_daily.dropna().copy()
X_daily = model_data_daily.drop(columns=['target_fwd_ret_5d'])
y_daily = model_data_daily['target_fwd_ret_5d']
X_daily.head()
```

	ret_1d	log_ret_1d	mom_5d	vol_5d	mom_21d	vol_21d	mom_63d	vol_63
Date								
2024-10-01	-0.029142	-0.029575	-0.005102	0.018809	-0.012183	0.015860	0.028156	0.0147
2024-10-02	0.002520	0.002517	0.001811	0.018739	0.018001	0.014633	0.024791	0.0147
2024-10-03	-0.004895	-0.004907	-0.008131	0.018661	0.021825	0.014533	-0.001806	0.0145
2024-10-04	0.005007	0.004995	-0.004346	0.018875	0.019876	0.014501	-0.003325	0.0144
2024-10-07	-0.022531	-0.022789	-0.048541	0.015258	0.003940	0.015305	-0.029445	0.0147

Exercise. Explain why the following split is inappropriate for a time-series forecasting problem.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

Temporal Validation

For the daily regression problem, we report RMSE, MAE, and R^2. In finance it is often also useful to report a directional metric, such as sign accuracy.

The validation scheme should preserve temporal order, so we use a rolling time-series split rather than a random shuffle.

```
def evaluate_daily_model(model, X, y, n_splits=5):
    tscv = TimeSeriesSplit(n_splits=n_splits)
    rows = []

    for fold, (train_idx, test_idx) in enumerate(tscv.split(X), start=1):
        X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
        y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

        model.fit(X_train, y_train)
        pred = model.predict(X_test)

        rows.append({
            'fold': fold,
            'rmse': mean_squared_error(y_test, pred) ** 0.5,
            'mae': mean_absolute_error(y_test, pred),
            'r2': r2_score(y_test, pred),
            'sign_accuracy': (np.sign(pred) == np.sign(y_test)).mean(),
        })

    return pd.DataFrame(rows)


models_daily = {
    'Linear': LinearRegression(),
    'Ridge': Ridge(alpha=1.0),
    'RandomForest': RandomForestRegressor(n_estimators=300, random_state=42, min_samples_lea
    'XGBoost': XGBRegressor(objective='reg:squarederror', n_estimators=200, max_depth=3, lea
}

daily_summary = []
for name, model in models_daily.items():
    fold_scores = evaluate_daily_model(model, X_daily, y_daily)
    daily_summary.append({
        'model': name,
        'rmse': fold_scores['rmse'].mean(),
        'mae': fold_scores['mae'].mean(),
        'r2': fold_scores['r2'].mean(),
        'sign_accuracy': fold_scores['sign_accuracy'].mean(),
    })

pd.DataFrame(daily_summary).sort_values('rmse').reset_index(drop=True)
```

	model	rmse	mae	r2	sign_accuracy
0	XGBoost	0.046009	0.037688	-0.173891	0.484615
1	RandomForest	0.048995	0.040568	-0.309114	0.496154
2	Ridge	0.049480	0.040332	-0.453446	0.476923
3	Linear	0.100502	0.083416	-6.978766	0.492308

Example B: LOBSTER Data

We now turn to a limit order book example using a LOBSTER sample for AAPL. Unlike the daily example, these data arrive in event time. Each row corresponds to a message event together with a synchronized view of the top levels of the book.

For a teaching example this is enough to illustrate the structure of the problem. It is not enough to make strong empirical claims about production performance.

```
def locate_lobster_sample():
    candidates = [
        Path('/Users/harold/4. RA work/FACTOR_TRAINING_CHI/lobster_samples/AAPL_2012-06-21_5
        Path('/Users/harold/Documents/LOBSTER_SampleFile_AAPL_2012-06-21_5.zip'),
    ]

    for candidate in candidates:
        if candidate.is_dir():
            return candidate
        if candidate.is_file() and candidate.suffix == '.zip':
            extract_dir = Path('/Users/harold/4. RA work/Factor_Training_Eng/.cache') / canc
            extract_dir.mkdir(parents=True, exist_ok=True)
            with zipfile.ZipFile(candidate) as zf:
                zf.extractall(extract_dir)
            return extract_dir

    raise FileNotFoundError('Could not locate the AAPL LOBSTER sample directory or zip file.


LOBSTER_DIR = locate_lobster_sample()
msg_path = next(LOBSTER_DIR.glob('*message_5.csv'))
book_path = next(LOBSTER_DIR.glob('*orderbook_5.csv'))

msg_cols = ['time', 'type', 'order_id', 'size', 'price', 'direction']
book_cols = []
for level in range(1, 6):
    book_cols += [f'ask_{level}', f'ask_size_{level}', f'bid_{level}', f'bid_size_{level}']

msg = pd.read_csv(msg_path, header=None, names=msg_cols)
book = pd.read_csv(book_path, header=None, names=book_cols)
lob = pd.concat([msg, book], axis=1)
lob.head()
```

	time	type	order_id	size	price	direction	ask_1	ask_size_1	bid_1
0	34200.004241	1	16113575	18	5853300	1	5859400	200	5853300
1	34200.004261	1	16113584	18	5853200	1	5859400	200	5853300
2	34200.004447	1	16113594	18	5853100	1	5859400	200	5853300
3	34200.025552	1	16120456	18	5859100	-1	5859100	18	5853300
4	34200.025580	1	16120480	18	5859200	-1	5859100	18	5853300

5 rows × 26 columns

Event-Level Features

The best bid and best ask determine the top of the book. From these quantities one may compute the mid-price, the spread, and various imbalance measures.

OFI is slightly different. It uses changes in prices and queue sizes to summarize net order book pressure at the best levels.

```
def compute_ofi_one_level(bid_price, bid_size, ask_price, ask_size):
    bid_prev = bid_price.shift(1)
    ask_prev = ask_price.shift(1)
    bid_size_prev = bid_size.shift(1)
    ask_size_prev = ask_size.shift(1)

    bid_contrib = np.where(
        bid_price > bid_prev,
        bid_size,
        np.where(bid_price == bid_prev, bid_size - bid_size_prev, -bid_size_prev),
    )
    ask_contrib = np.where(
        ask_price < ask_prev,
        ask_size,
        np.where(ask_price == ask_prev, ask_size - ask_size_prev, -ask_size_prev),
    )
    return pd.Series(bid_contrib - ask_contrib, index=bid_price.index)


feat_lob = lob.copy()
feat_lob['mid'] = (feat_lob['bid_1'] + feat_lob['ask_1']) / 2
feat_lob['spread'] = feat_lob['ask_1'] - feat_lob['bid_1']
feat_lob['spread_bps'] = 10000 * feat_lob['spread'] / feat_lob['mid']

feat_lob['imbalance_l1'] = (
    feat_lob['bid_size_1'] - feat_lob['ask_size_1']
) / (
    feat_lob['bid_size_1'] + feat_lob['ask_size_1']
)

bid_depth_5 = feat_lob[[f'bid_size_{i}' for i in range(1, 6)]].sum(axis=1)
```

```
ask_depth_5 = feat_lob[[f'ask_size_{i}' for i in range(1, 6)]].sum(axis=1)
feat_lob['imbalance_l5'] = (bid_depth_5 - ask_depth_5) / (bid_depth_5 + ask_depth_5)

feat_lob['microprice'] = (
    feat_lob['ask_1'] * feat_lob['bid_size_1'] + feat_lob['bid_1'] * feat_lob['ask_size_1']
) / (feat_lob['bid_size_1'] + feat_lob['ask_size_1'])
feat_lob['microprice_dev_bp'] = 10000 * (feat_lob['microprice'] - feat_lob['mid']) / feat_lo

feat_lob['ofi_l1'] = compute_ofi_one_level(
    feat_lob['bid_1'], feat_lob['bid_size_1'], feat_lob['ask_1'], feat_lob['ask_size_1']
)

feat_lob['ofi_l5'] = sum(
    compute_ofi_one_level(
        feat_lob[f'bid_{level}'],
        feat_lob[f'bid_size_{level}'],
        feat_lob[f'ask_{level}'],
        feat_lob[f'ask_size_{level}'],
    )
    for level in range(1, 6)
)

feat_lob['signed_size'] = feat_lob['direction'] * feat_lob['size']
feat_lob[['time', 'mid', 'spread_bps', 'imbalance_l1', 'imbalance_l5', 'ofi_l1']].head()
```

	time	mid	spread_bps	imbalance_l1	imbalance_l5	ofi_l1
0	34200.004241	5856350.0	10.416044	-0.834862	-0.561216	NaN
1	34200.004261	5856350.0	10.416044	-0.834862	-0.544715	0.0
2	34200.004447	5856350.0	10.416044	-0.834862	-0.639344	0.0
3	34200.025552	5856200.0	9.904033	0.000000	-0.629104	-18.0
4	34200.025580	5856200.0	9.904033	0.000000	-0.505325	0.0

Resampling in Event Time

The LOBSTER data are not equally spaced in time, so we construct fixed 1-second bars. To avoid lookahead, each event is assigned to the second obtained by taking the floor of its event time. Within each second we retain the last state of the book and aggregate message-flow quantities.

```
feat_lob['time_floor'] = np.floor(feat_lob['time']).astype(int)
feat_lob['time_str'] = pd.to_timedelta(feat_lob['time'], unit='s')

bars_lob = (
    feat_lob.groupby('time_floor', sort=True)
    .agg(
        time=('time', 'last'),
        bid_1=('bid_1', 'last'),
        bid_size_1=('bid_size_1', 'last'),
        ask_1=('ask_1', 'last'),
```

```
        ask_size_1=('ask_size_1', 'last'),
        mid=('mid', 'last'),
        spread=('spread', 'last'),
        spread_bps=('spread_bps', 'last'),
        imbalance_l1=('imbalance_l1', 'last'),
        imbalance_l5=('imbalance_l5', 'last'),
        microprice_dev_bp=('microprice_dev_bp', 'last'),
        ofi_l1=('ofi_l1', 'sum'),
        ofi_l5=('ofi_l5', 'sum'),
        signed_size=('signed_size', 'sum'),
        msg_count=('type', 'size'),
    )
    .reset_index()
)

bars_lob['time_sec'] = bars_lob['time_floor']
bars_lob['time_str'] = pd.to_timedelta(bars_lob['time_sec'], unit='s')
bars_lob['mid_next'] = bars_lob['mid'].shift(-1)
bars_lob['target_up_1s'] = (bars_lob['mid_next'] > bars_lob['mid']).astype(int)
bars_lob = bars_lob.dropna().copy()
bars_lob.head()
```

	time_floor	time	bid_1	bid_size_1	ask_1	ask_size_1	mid	spread
0	34200	34200.918221	5857400	100	5858700	100	5858050.0	1300
1	34201	34201.840852	5854700	18	5857700	18	5856200.0	3000
2	34202	34202.756909	5854700	100	5856800	18	5855750.0	2100
3	34203	34203.954770	5854500	18	5857100	100	5855800.0	2600
4	34204	34204.894471	5855000	18	5856800	900	5855900.0	1800

Temporal Validation for LOB Features

This second problem is a classification task. Because the target classes are often imbalanced, raw accuracy can be misleading. We therefore report balanced accuracy and AUC.

```
feature_cols_lob = [
    'spread_bps',
    'imbalance_l1',
    'imbalance_l5',
    'microprice_dev_bp',
    'ofi_l1',
    'ofi_l5',
    'signed_size',
    'msg_count',
```

```
]

X_lob = bars_lob[feature_cols_lob]
y_lob = bars_lob['target_up_1s']


def evaluate_lob_model(model, X, y, n_splits=5):
    tscv = TimeSeriesSplit(n_splits=n_splits)
    rows = []

    for fold, (train_idx, test_idx) in enumerate(tscv.split(X), start=1):
        X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
        y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

        model.fit(X_train, y_train)
        pred = model.predict(X_test)

        if hasattr(model, 'predict_proba'):
            prob = model.predict_proba(X_test)[:, 1]
        else:
            prob = model.predict(X_test)

        rows.append({
            'fold': fold,
            'balanced_accuracy': balanced_accuracy_score(y_test, pred),
            'auc': roc_auc_score(y_test, prob),
        })

    return pd.DataFrame(rows)


models_lob = {
    'Logistic': LogisticRegression(max_iter=500),
    'RandomForest': RandomForestClassifier(n_estimators=300, random_state=42, min_samples_le
    'XGBoost': XGBClassifier(
        objective='binary:logistic',
        eval_metric='logloss',
        n_estimators=200,
        max_depth=3,
        learning_rate=0.05,
        subsample=0.9,
        colsample_bytree=0.9,
        random_state=42,
    ),
}

lob_summary = []
for name, model in models_lob.items():
    fold_scores = evaluate_lob_model(model, X_lob, y_lob)
    lob_summary.append({
        'model': name,
        'balanced_accuracy': fold_scores['balanced_accuracy'].mean(),
        'auc': fold_scores['auc'].mean(),
    })

pd.DataFrame(lob_summary).sort_values('auc', ascending=False).reset_index(drop=True)
```

	model	balanced_accuracy	auc
0	XGBoost	0.510001	0.575260
1	RandomForest	0.509183	0.567462
2	Logistic	0.504085	0.557411

What Makes a Feature Useful?

A useful feature need not have a large marginal correlation with the target. More generally, a feature is attractive when it satisfies several criteria at once.

1. It carries predictive information.
2. It remains reasonably stable across time or regimes.
3. It is interpretable.
4. It can be computed in real time.
5. It survives practical constraints such as trading cost, latency, or limited data availability.

This is why feature engineering is better viewed as a modelling discipline than as a list of formulas.

Exercises

Exercise 1. Add rolling moving-average ratios to the daily feature set. Re-run the daily evaluation and compare the results to those obtained from the baseline set of return, momentum, volatility, and drawdown features.

Exercise 2. In a daily equity example, explain why accounting variables can create timing problems even when they appear to be aligned by date.

Exercise 3. In a LOBSTER example, compare level-1 imbalance and OFI. Which is easier to interpret economically? Which appears more useful empirically?

Exercise 4. Suppose that the LOBSTER data are aggregated using the ceiling of the event time rather than the floor. Explain why this may create leakage.

References

1. Cont, Kukanov, and Stoikov (2013), *The Price Impact of Order Book Events*.
2. Gould, Porter, Williams, McDonald, Fenn, and Howison (2013), *Limit Order Books*.
3. LOBSTER documentation and sample files.
4. Lopez de Prado (2018), *Advances in Financial Machine Learning*.